Memoria Transaccional

Programación Concurrente y Paralela, Clase práctica 6

Segundo cuatrimestre 2026

1. Instalación de Haskell y paquetes

Antes de empezar, necesitamos tener instalado Haskell. En esta sección vemos qué hay que instalar, cómo se agregan los paquetes que vamos a usar y cómo se ejecutan los programas.

1.1. Qué hay que instalar

Vamos a necesitar dos herramientas:

- **GHC** (*Glasgow Haskell Compiler*), el compilador de Haskell. Trae además `ghci`, el intérprete interactivo, y `runghc`, que permite ejecutar un archivo `.hs` sin compilarlo antes.
- **Cabal**, la herramienta que descarga e instala paquetes (bibliotecas) de Haskell.

La forma recomendada de instalar las dos es con **GHCup**, un instalador que se encarga de bajar las versiones adecuadas y de dejarlas configuradas. En Linux, macOS o WSL2 (Windows con Linux) se instala ejecutando en una terminal, como usuario normal (no como `root`):

```
curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

En Windows, el comando para PowerShell está en la página de GHCup, `https://www.haskell.org/ghcup/`, y también hay que ejecutarlo como usuario normal (no como administrador).

El instalador hace algunas preguntas. Alcanza con aceptar las opciones por defecto, en particular la de agregar los programas al `PATH`. También ofrece instalar HLS (*Haskell Language Server*), que no es necesario pero es útil si usamos un editor como VS Code con la extensión de Haskell, porque muestra los tipos y los errores mientras escribimos. En Linux, puede que antes haya que instalar algunas dependencias del sistema: la lista para cada distribución está en la página de requisitos de GHCup.

Al terminar, abrimos una terminal nueva y verificamos que todo quedó instalado:

```
ghc --version
cabal --version
```

1.2. Paquetes y módulos

En Haskell, el código de las bibliotecas se organiza en *módulos*, que son los que importamos con `import`. Los módulos, a su vez, se distribuyen en *paquetes*, que son lo que se instala. Los módulos que

vamos a usar son:

Módulo	Paquete	¿Viene con GHC?
`Control.Concurrent`	`base`	Sí
`System.IO`	`base`	Sí
`Control.Concurrent.STM`	`stm`	Sí
`Control.Concurrent.Async`	`async`	No
`System.Random`	`random`	No

El paquete `base` está siempre disponible. Para los demás, la forma más simple de usarlos fuera de un proyecto es instalarlos con `cabal install --lib`. Recomendamos instalarlos todos juntos:

```
cabal update
cabal install --lib stm async random
```

El primer comando actualiza la lista de paquetes disponibles en Hackage, el repositorio de paquetes de Haskell. El segundo los descarga, los compila y los deja disponibles para `ghc`, `ghci` y `runghc`.

Nota. ¿Por qué instalar `stm` si ya viene con GHC? Porque `cabal install --lib` registra los paquetes instalados en un *archivo de entorno* (en Linux, `~/.ghc/<arquitectura>-<versión>/environments/default`), y a partir del momento en que ese archivo existe, GHC sólo deja usar los paquetes que estén listados ahí, más `base`. Entonces, si instalamos sólo `async`, el paquete `stm` queda "escondido" y obtenemos un error como este:

```
Could not load module 'Control.Concurrent.STM'
It is a member of the hidden package 'stm-2.5.1.0'.
```

La solución es agregarlo con `cabal install --lib stm`. Lo mismo vale para cualquier otro paquete que venga con GHC y deje de encontrarse después de instalar otro. Además, el archivo de entorno depende de la versión de GHC: si en algún momento cambiamos de versión, hay que volver a instalar los paquetes.

1.3. Ejecutar los programas

Hay tres formas de ejecutar un archivo `.hs`:

- **Con el intérprete**: `ghci archivo.hs` abre el intérprete con el archivo cargado (también se puede abrir `ghci` y cargarlo con `:l archivo.hs`). Ahí podemos evaluar cualquier función del archivo, por ejemplo escribiendo `main`. Si modificamos el archivo, lo recargamos con `:r`, y para salir usamos `:q`.
- **Sin compilar**: `runghc archivo.hs` ejecuta la función `main` del archivo.
- **Compilando**: `ghc archivo.hs` genera un ejecutable (`archivo` en Linux y macOS, `archivo.exe` en Windows), que después corremos como cualquier programa.

Hay un detalle a tener en cuenta para la práctica: con `ghci`, con `runghc` y con un ejecutable compilado de la forma anterior, el programa usa un solo núcleo del procesador. Los threads que creemos se van a intercalar entre sí (y por lo tanto todos los problemas de concurrencia pueden aparecer), pero nunca van a correr en paralelo. Para que un ejecutable use varios núcleos, hay que compilarlo con la opción `-threaded` e indicarle al ejecutarlo cuántos núcleos usar:

```
ghc -threaded archivo.hs
./archivo +RTS -N4
```

Todo lo que viene después de `+RTS` son opciones para el *runtime* de Haskell, y `-N4` le indica que use 4 núcleos (con `-N` solo, usa todos los que tenga la máquina). Algunos errores de sincronización aparecen mucho más seguido cuando los threads corren realmente en paralelo, así que conviene probar los programas de las dos formas.

2. Mónada `IO` y *do notation*

Para esta práctica utilizamos el lenguaje de programación funcional Haskell, ya que nos provee una abstracción cómoda para manejarnos con memoria transaccional. A diferencia de las funciones puras de Haskell que creamos en otras materias, aquí crearemos threads y haremos cómputos con efectos e interacción con la entrada/salida en distintos momentos de nuestro cómputo. La manera de manejar esto es por medio de la mónada `IO`.

A efectos de esta materia, lo básico que debemos conocer sobre las mónadas es que toda mónada es algún tipo paramétrico `M a` (donde `a` es algún tipo). Lo que caracteriza a una mónada es la definición de dos funciones: `return` y *bind* (operador `>>=`):

- La función `return` tiene tipo `a -> M a`, y es la manera de tomar un valor del tipo `a` y darle el contexto que lo haga `M a`.
- La función *bind* (`>>=`) tiene tipo `M a -> (a -> M b) -> M b`. Representa la secuencia de eventos: Teniendo nuestro valor con contexto `x` del tipo `M a`, queremos aplicarle la función `f :: a -> M b` al valor de tipo `a` en `x`, y combinarlo con el contexto existente en `x` para generar un nuevo valor `y` de tipo `M b`.

Hay tipos paramétricos que ya habrán usado, como `Maybe` y las listas, que tienen implementadas estas funciones (el uso de `Maybe` como mónada es para modelar cómputos que pueden fallar, es decir, el uso de `>>=` permite componer funciones que individualmente pueden devolver `Nothing`; mientras que el de las listas es el no determinismo, es decir, `>>=` compone funciones con múltiples resultados posibles para alcanzar una lista de todas las posibilidades para la composición), pero nos interesará puntalmente `IO` (e introduciremos también `STM`).

`IO` nos permite componer funciones que pueden tener un efecto de entrada/salida (ejemplos clásicos: impresión en pantalla, lectura de archivos). Es decir, una función que devuelve un valor de tipo `IO a` puede (o no) tener un efecto colateral. Hay casos en los que la función en sí no devolverá nada, en ese caso su tipo de salida es `IO ()` (El `()` es el tipo *unit*, que tiene un único valor que es `"()"`). Como `IO` es mónada, contamos con `>>=` que nos permite componer distintas funciones de IO algo a IO otra cosa. Tomemos por ejemplo la siguiente porción de código:

```
duplicador = readLn >>= \n ->
  return (2 * n) >>= \m ->
  print ("El valor duplicado es" ++ (show m))
```

Lo que estamos haciendo aquí es tomar la función `readLn` que devuelve un `IO a` (en este caso deduciendo que el tipo `a` es numérico), y le pasa el valor de tipo `a` en esa `IO a` a una función anónima que toma un valor `n` de algún tipo numérico y hace `2 * n`. Como hacer eso da un valor de un

tipo numérico y para componer funciones con `>>=` necesitamos que el valor esté en el contexto `IO`, agregamos `return`, haciendo que la expresión sea `IO` de un número. Este número termina siendo el parámetro `m` de la función que lo toma, le aplica `show`, y lo imprime junto con el texto del valor duplicado. El tipo de salida de esta última sentencia `print` es `IO ()`.

Haskell provee un estilo de notación llamado *do notation*, que nos permite escribir el código de una manera que lo hace ver similar a una implementación imperativa:

```
duplicadorDo = do
  n <- readLn
  m <- return (2 * n)
  print ("El valor duplicado es" ++ (show m))
```

La función actúa de la misma manera (la *do notation* es sólo "syntax sugar"), pero ahora, los valores que correspondían a los parámetros de las funciones anónimas quedan escritos como si fueran "asignaciones" de variables.

Cuando queremos hacer un programa en Haskell que pueda compilar a un ejecutable, definimos en el archivo `.hs` una constante main con tipo `main :: IO ()`, que realiza su cómputo como una composición de funciones IO encadenadas con `>>=`.

2.1. Concurrencia: `fork`

Cuando queremos que nuestro programa concurrente tenga múltiples threads, invocamos alguna variante de la función `fork`, que va a crear un nuevo hilo de ejecución. El paquete `Control.Concurrent` (que debería estar instalado como parte de la distribución de Haskell) nos proporciona `forkOS` (que crea un thread del sistema operativo) y `forkIO` (que crea un thread manejado internamente por el runtime). Ambas tienen tipo `IO () -> IO ThreadId`. Es decir, en el contexto de una composición `IO` (o un bloque `do`) no toman un valor de entrada y devuelven un `threadId` en el contexto `IO`. El `IO ()` que toman es justamente la expresión que se evaluará en el thread nuevo.

Podemos probar el siguiente ejemplo:

```
variosPrints = do
  forkIO (print "Hola A")
  forkIO (print "Hola B")
  forkIO (print "Hola C")
  print("Holas")
```

De correrlo, posiblemente noten que la impresión de caracteres queda entremezclada. En efecto, en Haskell por default la impresión termina siendo caracter a caracter. Para tener impresiones por línea, podemos importar `System.IO` y agregar la siguiente línea al principio de nuestro bloque de código de tipo `IO ()`:

```
  hSetBuffering stdout LineBuffering
```

3. Mónada STM y acciones atómicas

Si queremos comunicar los threads que creamos, vamos a necesitar compartir valores. En la dinámica que conocemos de Haskell no guardamos valores en variables en el sentido tradicional, por lo que tenemos que incorporar elementos nuevos. Una vez que importemos el paquete `Control.Concurrent.STM`, vamos a contar con los tipos:

- `TVars`, las variables transaccionales. Allí guardaremos valores que podrán ser leídos desde múltiples threads.
- La mónada `STM`. Al igual que `IO`, tendremos no sólo `STM ()` sino `STM a` para cualquier tipo `a`. La idea es que el comportamiento en el entorno de memoria transaccional (que, por ejemplo, interactuará con las variables `TVar`) será una composición de funciones por medio de `>>=` (o un bloque `do`).

Las acciones básicas con las variables transaccionales son:

- `newTVar :: a -> STM (TVar a)`, que crea una variable transaccional de tipo `a` con el valor inicial que se le pase por parámetro.
- `readTVar :: TVar a -> STM a`, que toma una variable transaccional y devuelve su valor actual.
- `writeTVar :: TVar a -> a -> STM ()`, que dada una variable transaccional y un valor del tipo que esta almacena, guarda ese valor en la variable
- `modifyTVar' :: TVar a -> (a -> a) -> STM ()`, que dada una variable transaccional y una función en el tipo de la variable, aplica la función al valor actual de la variable (la versión con `'` es *eager* y se recomienda ante la que no lo tiene, que es *lazy*).

Como podemos ver, siempre nos mantenemos en el contexto de `STM` (por lo que no podemos hacer `print`, que devuelve un `IO ()`, por ejemplo). Si estamos en un contexto `IO` y queremos realizar acciones de `STM`, la clave está en la función `atomically`. El tipo de `atomically` es `STM a -> IO a`, dada una expresión de tipo `STM a`, obtiene el `IO a` donde el valor es el mismo. Luego, cuando queremos ejecutar código `STM` lo haremos dentro del parámetro de `atomically`, lo cual implicará que Haskell se va a encargar que toda la expresión se evalúe en conjunto como una acción.

Excepcionalmente, existen dos funciones que trabajan con `TVars` directamente en un bloque `IO`: creación (`newTVarIO :: a -> IO (TVar a)`) y lectura (`readTVarIO :: TVar a -> IO a`). Sin embargo, es importante tomar en cuenta que como su uso no es dentro de una expresión en un `atomically`, no tenemos garantía de que el valor que hayamos leído con un `readTVarIO` se preserve en una operación posterior.

Consideremos un programa de ejemplo donde tenemos un bloque `IO` y un bloque `STM`:

```
manipulaSTM  = do
  var1 <-newTVarIO 10
  var2 <-newTVarIO 20
  atomically (do
    v1 <- readTVar var1
    v2 <- readTVar var2
    writeTVar var2 v1
    writeTVar var1 v2
    )
  v1 <- readTVarIO var1
```

```
  v2 <- readTVarIO var2
  print("Valor de var1: " ++ (show v1))
  print("Valor de var2: " ++ (show v2))
```

El bloque `manipulaSTM` intercambia atómicamente los valores de `var1` y `var2`. Notar que es un bloque `do IO` que en un momento hace `atomically` de un bloque `do STM`. Por ello, en el bloque interno se puede usar `readTVar` y `writeTVar` mientras que en el externo se usa `newTVarIO` y `readTVarIO`.

4. Como funcionan las transacciones, conflictos y las esperas

4.1. Cómo se ejecuta una transacción

Cuando un thread ejecuta `atomically t`, no realiza una operacion bloqueante. Ejecuta `t` de manera *optimista*, suponiendo que ningún otro thread va a interferir, y mientras tanto va anotando en su *log* (de la transacción) todas las `TVar`s que lee, con el valor que vio, y todas las que escribe, con el valor nuevo. Las escrituras no impactan en las `TVar`s mientras la transacción se está ejecutando: quedan sólo en el log. Es decir, no se vuelvan a memoria durante la transaccion, como vamos a ver despues, solamente se vuelvan al final, cuando se hace un commit exitoso. Por eso ningún otro thread puede ver un estado intermedio de la transacción. Si la misma transacción vuelve a leer una `TVar` que ya escribió, obtiene el valor nuevo que tiene anotado en su log.

Al terminar, la transacción intenta hacer **commit**. Para eso verifica que todas las `TVar`s que leyó sigan teniendo el valor que vio:

- Si es así, todas sus escrituras impactan juntas, de una sola vez. Para el resto de los threads es como si la transacción entera hubiera ocurrido en ese instante.
- Si alguna cambió (porque otro thread confirmó una transacción que la escribió), hay un *conflicto*. El log se descarta, como si la transacción nunca se hubiera ejecutado, y se vuelve a ejecutar desde el principio, inmediatamente, esta vez con los valores nuevos.

Veamos un ejemplo. Supongamos que dos threads, A y B, ejecutan a la vez `atomically (incrementar contador)`, donde `contador` empieza en 0:

```
incrementar var = do
  valor <- readTVar var
  writeTVar var (valor + 1)
```

Los dos leen 0 y anotan en su log que van a escribir 1. A se confirma primero: `contador` sigue valiendo 0, así que su escritura impacta y `contador` pasa a valer 1. Cuando B intenta confirmarse, ve que `contador` ya no vale 0, que es lo que había leído. Hay un conflicto, así que B descarta su log y se vuelve a ejecutar: ahora lee 1, escribe 2 y se confirma. El resultado final es 2, y no se perdió ningún incremento.

Que una transacción se pueda ejecutar varias veces antes de confirmarse es la razón por la que dentro de un bloque `STM` no se pueden hacer acciones de `IO`: si pudiéramos imprimir en pantalla dentro de una transacción, el mensaje aparecería una vez por cada intento, y además no habría forma de "deshacerlo" al descartar el log. En Haskell esto no es sólo una recomendación, lo garantizan los tipos: `print` devuelve `IO ()`, y en un bloque `do` de `STM` sólo se pueden encadenar acciones `STM`.

4.2. Esperar una condición: `retry`

Hasta acá, una transacción siempre puede terminar. Pero muchas veces una operación no tiene sentido hasta que se cumpla alguna condición, como sacar un elemento de un buffer vacío o decrementar un contador que está en 0. En esos casos no queremos que la operación falle, sino que espere a que la condición se cumpla. Para eso está la constante `retry :: STM a`. Tomemos como ejemplo una función que decrementa el valor de una `TVar`, pero sólo si no es 0:

```
decrementarSiPuede var = do
  actual <- readTVar var
  if (actual == 0)
    then retry
    else writeTVar var (actual - 1)
```

Cuando la ejecución llega a un `retry`, la transacción se aborta. Igual que en un conflicto, se descartan todos los cambios anotados en el log, así que es como si nunca se hubiera ejecutado. La diferencia está en *cuándo* se vuelve a intentar:

- Después de un conflicto, la transacción se vuelve a ejecutar enseguida, porque ahora, con los valores nuevos, es probable que pueda terminar.
- Después de un `retry`, volver a ejecutarla enseguida no serviría de nada: leería los mismos valores y llegaría otra vez al `retry`. Por eso el thread se duerme. El runtime usa el log para saber qué `TVar`s leyó la transacción antes de llegar al `retry`, y recién cuando otro thread modifica alguna de ellas, despierta al thread y la transacción se vuelve a ejecutar desde el principio. De esta forma no hay *busy waiting*: un thread que espera no consume procesador.

Como nota, observar que `decrementarSiPuede` es de tipo `(Eq a, Num a) => TVar a -> STM ()`. Es decir, es una transacción y no una acción de `IO`, lo que indica que para ejecutarla hay que usarla dentro de un `atomically`. Puede ser directamente, como en `atomically (decrementarSiPuede var)`, o como parte de una transacción más grande que la combine con otras operaciones. En ese caso, si `decrementarSiPuede` llega al `retry`, se descarta la transacción entera, incluidos los cambios que hayan hecho las operaciones anteriores.

4.3. `check` y `orElse`

Hay dos funciones que simplifican el uso de `retry`.

La primera es `check :: Bool -> STM ()`. Esta función es como una aserción: toma una expresión booleana, y si es verdadera continúa con la siguiente operación, y si es falsa hace `retry`. Con `check`, el ejemplo del decremento queda:

```
decrementarSiPuedeCheck var = do
  actual <- readTVar var
  check (actual > 0)
  writeTVar var (actual - 1)
```

La condición que se le pasa a `check` es la que tiene que cumplirse para poder seguir, es decir, la contraria a la del `if` de la versión anterior.

La segunda es `orElse :: STM a -> STM a -> STM a`, que permite ofrecer una alternativa. Toma dos transacciones e intenta llevar a cabo la primera. Si tiene éxito, el resultado es el de la primera. Si llega a un `retry`, se descartan sus cambios y se prueba con la segunda. Si la segunda también llega a un `retry`, toda la transacción hace `retry`, y el thread se duerme hasta que cambie alguna de las `TVar`s que leyó cualquiera de las dos. Por ejemplo, para decrementar alguno de dos contadores, dándole prioridad al primero:

```
decrementarAlguno var1 var2 =
  decrementarSiPuede var1 `orElse` decrementarSiPuede var2
```

Las comillas invertidas permiten usar una función de dos parámetros como operador, entre sus argumentos. Es lo mismo que escribir `orElse (decrementarSiPuede var1) (decrementarSiPuede var2)`.

5. Algunas operaciones adicionales

Dejamos algunas otras funciones que pueden ser de utilidad para hacer pruebas o resolver los ejercicios:

- `threadDelay :: Int -> IO ()`: Para demorar la ejecución de threads. Como sabemos, no es una manera fiable de hacer sincronización, pero puede sernos útil. El entero está medido en microsegundos (10^{-6} segundo).
- `data`: Esto no es nuevo del manejo de concurrencia, pero recordemos que podemos definir tipos nuevos con la keyword `data`. Esto nos permite tener por ejemplo `data MiTipo = MiTipo {campo1 :: TVar Int, campo2 :: TVar Bool}`; con los campos declarados de esa manera, al querer referirnos a una `TVar` interna de un tipo, el nombre del campo es a su vez la función proyectora: si `tip` es un valor de tipo `MiTipo`, `readTVar (campo1 tip)` devuelve el valor en `campo1` de `tip`. También se puede usar `type`, que es un renombre de tipo (útil para mejorar la legibilidad del código).
- Es posible que nuestros programas tengan threads que están en bucles que corren por siempre, como en un `while(true)`. Una forma de escribir esto en Haskell es con una expresión recursiva sin caso base. Quedaría:

  ```
  eterno = loop
    where
      loop = do
        print "Hola"
        threadDelay 1000000
        loop
  ```

- El módulo `Control.Concurrent.Async` ofrece la posibilidad de lanzar un thread y esperar un valor de retorno del mismo. No es estrictamente necesario para resolver los ejercicios de la práctica, pero lo usamos en los ejemplos del final para que el thread principal espere a que terminen los demás.
- Otro módulo con el que se puede probar es `System.Random`, para generar números al azar (por ejemplo, demoras aleatorias con `threadDelay`).

6. Ejemplos

Veamos dos ejemplos. El primero es un ejercicio sencillo para ver cómo encajan todas las piezas que presentamos hasta ahora: threads, `TVar`s, `atomically` y `check`. El segundo es una estructura de datos concurrente, que vamos a comparar con las implementaciones con locks que vimos en la teórica de listas.

6.1. La largada de una carrera

> En una carrera participan N corredores. Cada corredor tarda un tiempo distinto en prepararse y, cuando está listo, avisa y espera la señal de largada. El juez espera a que todos los corredores estén listos y recién entonces da la largada. Cada corredor, al llegar a la meta, se anota en la lista de llegadas y muestra por pantalla en qué puesto llegó. Cuando todos terminaron, el programa muestra el orden de llegada completo.

Vamos a modelar a cada corredor como un thread, y al juez como el thread principal (`main`).

El estado compartido

Antes de escribir código conviene preguntarse qué información tienen que compartir los threads. Cada una de esas informaciones va a ser una `TVar`:

- `ready :: TVar Int`, la cantidad de corredores que ya están listos. La escriben los corredores y la lee el juez.
- `start :: TVar Bool`, que indica si ya se dio la largada. La escribe el juez y la leen los corredores.
- `arrivals :: TVar [Int]`, los números de los corredores que ya llegaron, en orden de llegada.

En Haskell no hay variables globales como en Java o C, así que estas `TVar`s las va a crear `main` y se las va a pasar como parámetro a cada corredor.

Además del estado compartido, el juez tiene que poder esperar a que cada corredor termine. Para eso vamos a lanzar los threads con el paquete `Control.Concurrent.Async` (que mencionamos en la sección anterior) en lugar de `forkIO`.

El programa completo

Primero veamos el programa entero, y después lo recorremos parte por parte.

```
import Control.Concurrent
import Control.Concurrent.STM
import Control.Concurrent.Async
import System.IO

runner :: TVar Int -> TVar Bool -> TVar [Int] -> Int -> IO ()
runner ready start arrivals i = do
  threadDelay (i * 100000)
  atomically (modifyTVar' ready (+1))
  atomically (do
```

```
    started <- readTVar start
    check started)
  threadDelay ((5 - i) * 200000)
  position <- atomically (do
    finished <- readTVar arrivals
    writeTVar arrivals (finished ++ [i])
    return (length finished + 1))
  putStrLn ("Corredor " ++ show i ++ " llegó en el puesto " ++ show position)

main :: IO ()
main = do
  hSetBuffering stdout LineBuffering
  let n = 4
  ready    <- newTVarIO 0
  start    <- newTVarIO False
  arrivals <- newTVarIO []
  runners  <- mapM (\i -> async (runner ready start arrivals i)) [1..n]
  atomically (do
    r <- readTVar ready
    check (r == n))
  putStrLn "¡Largaron!"
  atomically (writeTVar start True)
  mapM_ wait runners
  podium <- readTVarIO arrivals
  print podium
```

Al ejecutarlo obtenemos:

```
¡Largaron!
Corredor 4 llegó en el puesto 1
Corredor 3 llegó en el puesto 2
Corredor 2 llegó en el puesto 3
Corredor 1 llegó en el puesto 4
[4,3,2,1]
```

El corredor

Empecemos por el código del corredor, de a una línea (o de a un bloque) por vez.

```
runner :: TVar Int -> TVar Bool -> TVar [Int] -> Int -> IO ()
runner ready start arrivals i = do
```

El corredor recibe las tres `TVar`s compartidas y su número `i`. El tipo de salida es `IO ()`: es la acción que va a ejecutar el thread del corredor, y no devuelve ningún resultado. Todo el cuerpo es un bloque `do` de `IO`.

```
  threadDelay (i * 100000)
```

Simulamos el tiempo de preparación: el corredor `i` tarda `i` décimas de segundo. Recordemos que `threadDelay` no sirve para sincronizar, acá solamente representa que el corredor está ocupado haciendo algo.

```
atomically (modifyTVar' ready (+1))
```

El corredor avisa que está listo sumando uno a `ready`. Hay dos cosas para notar:

- `modifyTVar' ready (+1)` tiene tipo `STM ()`, y estamos en un bloque `IO`. Por eso hace falta `atomically`, que la convierte en una acción `IO ()`. Aunque la transacción tenga una sola operación, no hay otra forma de ejecutar código `STM` desde `IO`.
- Incrementar es leer el valor, sumarle uno y escribirlo. Si dos corredores se ponen listos al mismo tiempo y esas operaciones se intercalan, se podría perder un incremento (es el mismo problema del contador compartido que vimos en clases anteriores). Al estar dentro de una transacción, eso no puede pasar: si dos transacciones entran en conflicto, una de ellas se vuelve a ejecutar y ve el valor actualizado.

```
atomically (do
  started <- readTVar start
  check started)
```

Ahora el corredor espera la largada. Esta transacción lee `start` y hace `check` sobre ese valor. Si todavía es `False`, `check` hace `retry`. Como vimos antes, el reintento no es inmediato: el runtime registra que esta transacción leyó `start` y duerme al thread hasta que alguien escriba esa `TVar`. Es decir, el corredor no hace *busy waiting*. Cuando el juez escriba `True` en `start`, la transacción se vuelve a ejecutar, esta vez el `check` pasa y el corredor sigue.

Notar que avisar que está listo y esperar la largada son dos transacciones separadas, y no una sola. Esto no es casual, lo discutimos más abajo.

```
threadDelay ((5 - i) * 200000)
```

Simulamos el tiempo que tarda en correr. Elegimos los tiempos para que el último en prepararse sea el más rápido, así el orden de llegada es distinto del orden en que se prepararon (para eso asumimos que hay a lo sumo 4 corredores).

```
position <- atomically (do
  finished <- readTVar arrivals
  writeTVar arrivals (finished ++ [i])
  return (length finished + 1))
```

Esta es la parte más interesante. El corredor llega a la meta y, en una única transacción:

1. lee la lista de los que llegaron antes (`finished`),
2. se agrega al final de esa lista,
3. devuelve su puesto, que es la cantidad de corredores que llegaron antes más uno.

El `return` de la última línea es el `return` de la mónada `STM`: toma el `Int` y lo pone en el contexto `STM`, así que todo el bloque `do` interno tiene tipo `STM Int`. Entonces `atomically` de ese bloque tiene tipo `IO Int`, y con `position <-` obtenemos el `Int` dentro del bloque `IO` externo. Este es el patrón habitual para sacar información de una transacción: la transacción la devuelve como resultado.

Lo importante es que la lectura de la lista y la escritura están en la *misma* transacción. Si dos corredores llegan al mismo tiempo, no pueden ver los dos la misma lista y anotarse en el mismo

puesto: alguna de las dos transacciones va a detectar el conflicto y se va a volver a ejecutar, esta vez viendo la lista que ya incluye al otro corredor.

```
  putStrLn ("Corredor " ++ show i ++ " llegó en el puesto " ++ show position)
```

Mostramos el puesto. Esto tiene que estar *afuera* del `atomically`: `putStrLn` devuelve `IO ()`, y dentro de un bloque `STM` solamente podemos usar acciones `STM`, así que el compilador no nos dejaría ponerlo adentro. Y está bien que así sea: una transacción se puede ejecutar varias veces antes de confirmarse, y no queremos que el mensaje se imprima una vez por cada intento. Por eso la transacción calcula y devuelve el puesto, y la impresión ocurre después, una sola vez.

El juez

El juez es el thread principal, es decir, `main`.

```
main :: IO ()
main = do
  hSetBuffering stdout LineBuffering
  let n = 4
```

La primera línea es la que vimos antes para que las impresiones de distintos threads no se mezclen caracter a caracter. Con `let` definimos la cantidad de corredores. Notar que usamos `let` y no `<-`: `4` es un valor común, no una acción `IO` de la cual haya que extraer un resultado.

```
  ready    <- newTVarIO 0
  start    <- newTVarIO False
  arrivals <- newTVarIO []
```

Creamos las tres `TVars` con sus valores iniciales: ningún corredor listo, la largada sin dar y nadie llegó. Usamos `newTVarIO` porque estamos en un bloque `IO`. No hace falta escribir los tipos: Haskell los deduce a partir de cómo se usan (por ejemplo, de la signatura de `runner`).

```
  runners  <- mapM (\i -> async (runner ready start arrivals i)) [1..n]
```

Lanzamos un thread por corredor. La función `async :: IO a -> IO (Async a)` es parecida a `forkIO`: recibe una acción y la ejecuta en un thread nuevo. La diferencia es lo que devuelve. En lugar de un `ThreadId`, devuelve un valor de tipo `Async a`, que funciona como un comprobante: más adelante lo podemos usar para esperar a que ese thread termine y obtener el resultado de su acción (de tipo `a`). En nuestro caso la acción es de tipo `IO ()`, así que cada corredor es un `Async ()`.

La función `mapM` toma una función que devuelve una acción y una lista, ejecuta la acción para cada elemento de la lista, en orden, y devuelve la lista de resultados. Así, `runners` es una lista con los cuatro `Async ()`, uno por corredor. Es equivalente a escribir:

```
  r1 <- async (runner ready start arrivals 1)
  r2 <- async (runner ready start arrivals 2)
  r3 <- async (runner ready start arrivals 3)
  r4 <- async (runner ready start arrivals 4)
  let runners = [r1, r2, r3, r4]
```

Observar que `runner ready start arrivals 1` es un valor de tipo `IO ()` que *describe* lo que hace el corredor 1, pero no lo ejecuta. Quien lo ejecuta, en un thread nuevo, es `async`.

```
atomically (do
  r <- readTVar ready
  check (r == n))
```

El juez espera a que estén todos listos. Es el mismo mecanismo que usan los corredores para esperar la largada: si `r` todavía no llegó a `n`, `check` hace `retry` y el juez se duerme hasta que alguien modifique `ready`. Cada vez que un corredor se pone listo, el juez se despierta, vuelve a ejecutar la transacción y, si todavía faltan corredores, se vuelve a dormir.

```
putStrLn "¡Largaron!"
atomically (writeTVar start True)
```

El juez anuncia la largada y escribe `True` en `start`. Esa escritura es la que despierta a todos los corredores que estaban esperando. Imprimimos el mensaje antes de escribir `start` para que aparezca siempre antes que cualquier llegada.

```
mapM_ wait runners
podium <- readTVarIO arrivals
print podium
```

Finalmente, el juez espera a que todos terminen y muestra el orden de llegada. La función `wait :: Async a -> IO a` bloquea al thread que la llama hasta que termine el thread asociado a ese `Async`, y devuelve su resultado. Con `mapM_` la aplicamos a cada corredor, en orden, y descartamos los resultados (que son todos `()`). Cuando termina esa línea, sabemos que los cuatro corredores terminaron por completo. Si algún corredor hubiera terminado con una excepción, `wait` la relanza en el juez, así que el error no pasaría desapercibido.

Acá usamos `readTVarIO` sin problemas: dijimos que no hay garantía de que el valor leído se preserve, pero en este punto ya no queda nadie que vaya a modificar `arrivals`.

¿Por qué hace falta esperar? Porque cuando `main` termina, termina todo el programa, incluyendo los threads que todavía estén corriendo. Si el juez no esperara, el programa podría terminar antes de que los corredores lleguen (algo que ya podía pasar en el ejemplo de `variosPrints`).

¿Y no se podría esperar sin `Async`, con una transacción que haga `check` hasta que `arrivals` tenga `n` elementos? Probemos esa versión: el último corredor se anota en `arrivals` *antes* de imprimir su puesto. Entonces el juez se puede despertar, mostrar el podio y terminar el programa antes de que ese corredor llegue a imprimir. En una ejecución de esa versión obtuvimos:

```
¡Largaron!
Corredor 4 llegó en el puesto 1
Corredor 3 llegó en el puesto 2
Corredor 2 llegó en el puesto 3
[4,3,2,1]
Corredor 1 llegó en el puesto 4
```

Esta vez el corredor 1 llegó a imprimir justo antes de que terminara el programa, pero nada lo garantiza. La moraleja es que hay que esperar al evento que realmente nos interesa (que el corredor terminó todo lo que tenía que hacer) y no a uno que suele ocurrir cerca. Eso es justamente lo que

hace `wait`.

Un posible error

Juntar el aviso y la espera en una sola transacción. Podríamos pensar que el corredor puede avisar que está listo y esperar la largada en un mismo `atomically`:

```
atomically (do
  modifyTVar' ready (+1)
  started <- readTVar start
  check started)
```

Parece equivalente, pero no lo es. Cuando `check` hace `retry`, *todos* los cambios de la transacción se descartan, incluido el incremento de `ready`. Entonces `ready` nunca deja de valer 0: el juez espera a que estén todos listos, y los corredores esperan una largada que nunca llega. Es un deadlock. En este caso el runtime de Haskell se da cuenta de que ningún thread puede avanzar y corta el programa con el error `thread blocked indefinitely in an STM transaction`. La moraleja es que una transacción que espera no puede avisarle nada al resto: sus efectos sólo se vuelven visibles si la transacción termina.

6.2. Un conjunto como lista enlazada

En la teórica de listas vimos cómo implementar un conjunto de enteros con las operaciones `add`, `remove` y `contains` usando una lista simplemente enlazada y ordenada, y distintas formas de sincronizarla para que la usen varios threads a la vez: granularidad gruesa (un lock para toda la lista), granularidad fina (un lock por nodo, con *hand-over-hand*), sincronización optimista y sincronización *lazy*. Veamos cómo queda la misma estructura con STM, y qué tiene en común con cada una de esas versiones.

Para simplificar, el conjunto va a guardar directamente enteros (las claves), en lugar de objetos con su `hashCode`.

La representación

En Java, un nodo tenía una clave y una referencia `next` al nodo siguiente, que es lo único que se modifica cuando se agrega o se saca un elemento. En Haskell los valores no se modifican, así que la parte mutable del nodo tiene que ser explícita: el campo `next` va a ser una `TVar`.

```
data Node = Node Int (TVar Node) | Nil

type List = TVar Node
```

Un `Node` es o bien un nodo con su clave y su `next`, o bien `Nil`, que marca el final de la lista. La clave de un nodo no cambia nunca, lo único que cambia es el contenido de las `TVar`s. Es decir, cada `TVar Node` cumple el papel de un campo `next` de Java: es una referencia modificable a un nodo.

La lista en sí es una `TVar Node`, la que apunta al primer nodo. En la teórica se usaban dos nodos centinela, `head` y `tail`, para no tener que tratar aparte los casos del principio y del final de la lista. Acá ese papel lo cumplen la `TVar` de la lista (que funciona como el `next` de `head`) y `Nil` (que funciona como `tail`).

El invariante es el mismo que en la teórica: las claves están ordenadas de menor a mayor y no hay claves repetidas.

El programa completo

Como antes, primero veamos el programa entero, y después lo recorremos parte por parte.

```
import Control.Concurrent.STM
import Control.Concurrent.Async

data Node = Node Int (TVar Node) | Nil

type List = TVar Node

newList :: IO List
newList = newTVarIO Nil

find :: TVar Node -> Int -> STM (TVar Node)
find predNext key = do
  curr <- readTVar predNext
  case curr of
    Node k currNext | k < key -> find currNext key
    _                         -> return predNext

add :: List -> Int -> STM Bool
add list key = do
  predNext <- find list key
  curr <- readTVar predNext
  case curr of
    Node k _ | k == key -> return False
    _ -> do
      nodeNext <- newTVar curr
      writeTVar predNext (Node key nodeNext)
      return True

remove :: List -> Int -> STM Bool
remove list key = do
  predNext <- find list key
  curr <- readTVar predNext
  case curr of
    Node k currNext | k == key -> do
      next <- readTVar currNext
      writeTVar predNext next
      return True
    _ -> return False

contains :: List -> Int -> STM Bool
contains list key = do
  predNext <- find list key
  curr <- readTVar predNext
  case curr of
    Node k _ -> return (k == key)
    Nil      -> return False
```

```
toList :: List -> STM [Int]
toList predNext = do
  curr <- readTVar predNext
  case curr of
    Nil -> return []
    Node k currNext -> do
      rest <- toList currNext
      return (k : rest)

addAll :: List -> [Int] -> IO ()
addAll list keys = mapM_ (\k -> atomically (add list k)) keys

removeAll :: List -> [Int] -> IO ()
removeAll list keys = mapM_ (\k -> atomically (remove list k)) keys

main :: IO ()
main = do
  list <- newList
  adders <- mapM (\t -> async (addAll list [t, t + 4 .. 200])) [1 .. 4]
  mapM_ wait adders
  removers <- mapM (\t -> async (removeAll list [t, t + 4 .. 200])) [1, 2]
  mapM_ wait removers
  elems <- atomically (toList list)
  print elems
```

Crear la lista

```
newList :: IO List
newList = newTVarIO Nil
```

Una lista vacía es una `TVar` que apunta a `Nil`. Usamos `newTVarIO` para poder crearla directamente desde un bloque `IO`.

Buscar la posición

En la teórica, las tres operaciones empezaban con el mismo recorrido: avanzar con dos referencias, `pred` y `curr`, hasta que `curr` fuera el primer nodo con clave mayor o igual a la buscada.

```
pred = head;
curr = pred.next;
while (curr.key < key) {
  pred = curr;
  curr = curr.next;
}
```

Ese recorrido lo escribimos una sola vez, en la función `find`:

```
find :: TVar Node -> Int -> STM (TVar Node)
find predNext key = do
```

```
  curr <- readTVar predNext
  case curr of
    Node k currNext | k < key -> find currNext key
    _                         -> return predNext
```

En lugar de devolver `pred`, `find` devuelve `predNext`, la `TVar` que en Java sería `pred.next`. Es justamente lo que vamos a querer modificar después, y su contenido es `curr`. Veamos línea por línea:

- `curr <- readTVar predNext` lee el nodo al que apunta `predNext`, es decir, `curr = pred.next`.
- `case curr of` distingue según la forma de `curr`. La primera alternativa, `Node k currNext | k < key`, sólo se elige si `curr` es un nodo (y no `Nil`) *y además* su clave `k` es menor que la buscada. Lo que viene después de la `|` se llama *guarda*, y es una condición extra que tiene que cumplirse para que se elija esa alternativa. En ese caso todavía no llegamos y seguimos buscando desde el `next` de `curr`. Esto es el cuerpo del `while`: el `curr` actual pasa a ser el nuevo `pred`.
- La alternativa `_` es la de "cualquier otro caso": `curr` es `Nil` o es un nodo con clave mayor o igual. Ya llegamos, y devolvemos `predNext`.

Como en Haskell no hay `while`, la iteración se escribe con recursión, igual que el `loop` de la sección anterior.

Hay un detalle que va a ser importante más adelante: cada `readTVar` que hace `find` queda registrado en el log de la transacción. Así que cuando `find` termina, la transacción "leyó" todos los punteros `next` desde el principio de la lista hasta la posición buscada.

Agregar

```
add :: List -> Int -> STM Bool
add list key = do
  predNext <- find list key
  curr <- readTVar predNext
  case curr of
    Node k _ | k == key -> return False
    _ -> do
      nodeNext <- newTVar curr
      writeTVar predNext (Node key nodeNext)
      return True
```

Primero buscamos la posición empezando desde la `TVar` de la lista, y leemos `curr`. Si `curr` es un nodo con la misma clave, el elemento ya está y devolvemos `False` (en el patrón `Node k _`, el `_` indica que no nos interesa el `next` de ese nodo). Si no, creamos el nodo nuevo y lo enganchamos entre `pred` y `curr`, igual que en Java:

```
Node node = new Node(key);
node.next = curr;
pred.next = node;
```

`newTVar curr` crea el `next` del nodo nuevo, que ya apunta a `curr`. Después `writeTVar predNext (Node key nodeNext)` hace que `pred.next` apunte al nodo nuevo. Usamos `newTVar` y no `newTVarIO` porque estamos dentro de una transacción.

Sacar

```
remove :: List -> Int -> STM Bool
remove list key = do
  predNext <- find list key
  curr <- readTVar predNext
  case curr of
    Node k currNext | k == key -> do
      next <- readTVar currNext
      writeTVar predNext next
      return True
    _ -> return False
```

Si `curr` es un nodo con la clave buscada, lo desenganchamos: leemos su `next` y hacemos que `pred.next` apunte ahí. Es la línea `pred.next = curr.next` de Java, que acá ocupa dos líneas porque leer y escribir una `TVar` son dos operaciones separadas. Si no está, devolvemos `False`.

Pertenencia

```
contains :: List -> Int -> STM Bool
contains list key = do
  predNext <- find list key
  curr <- readTVar predNext
  case curr of
    Node k _ -> return (k == key)
    Nil      -> return False
```

El elemento está si `curr` es un nodo con la clave buscada. Si llegamos a `Nil`, no está.

Notar que las tres operaciones devuelven `STM Bool` y no `IO Bool`: no ejecutan nada por sí solas, sino que describen una transacción. El que las use decide dónde poner el `atomically`. Esto es lo mismo que pasaba con `decrementarSiPuede`, y más abajo vamos a ver por qué es tan útil.

Probándolo

Para ver el contenido de la lista, `toList` la recorre entera y arma una lista de Haskell con las claves:

```
toList :: List -> STM [Int]
toList predNext = do
  curr <- readTVar predNext
  case curr of
    Nil -> return []
    Node k currNext -> do
      rest <- toList currNext
      return (k : rest)
```

Como todo el recorrido está en una única transacción, el resultado es una foto consistente de la lista en algún instante, aunque haya otros threads modificándola.

```
addAll :: List -> [Int] -> IO ()
```

```
addAll list keys = mapM_ (\k -> atomically (add list k)) keys

removeAll :: List -> [Int] -> IO ()
removeAll list keys = mapM_ (\k -> atomically (remove list k)) keys
```

Estas dos funciones agregan (o sacan) una lista de claves, cada una en su propia transacción. Así, mientras un thread recorre sus claves, las operaciones de distintos threads se van intercalando.

```
main :: IO ()
main = do
  list <- newList
  adders <- mapM (\t -> async (addAll list [t, t + 4 .. 200])) [1 .. 4]
  mapM_ wait adders
  removers <- mapM (\t -> async (removeAll list [t, t + 4 .. 200])) [1, 2]
  mapM_ wait removers
  elems <- atomically (toList list)
  print elems
```

En la primera etapa, cuatro threads agregan a la vez los números del 1 al 200: el thread `t` agrega `t`, `t + 4`, `t + 8`, etc. (la expresión `[t, t + 4 .. 200]` es la lista que empieza en `t` y avanza de a 4 hasta 200). Esperamos a que terminen con `wait`, igual que en el ejemplo de la carrera. En la segunda etapa, dos threads sacan a la vez: el primero saca 1, 5, 9, ... y el segundo saca 2, 6, 10, ... Es decir, los dos threads van sacando nodos *adyacentes* al mismo tiempo, que es justamente el caso problemático que vimos en la teórica. Al final deberían quedar los números que dan resto 3 o 0 al dividirlos por 4, y efectivamente eso es lo que obtenemos:

```
[3,4,7,8,11,12,15,16,19,20,23,24,27,28,31,32,35,36,39,40, ...
```

Comparación con las versiones con locks

El código es el secuencial. Si comparamos `remove` con la versión secuencial en Java de la teórica, es el mismo algoritmo, renglón por renglón. No hay locks, ni validación, ni marcas de borrado. Lo único que cambió es que los `next` se leen y se escriben con `readTVar` y `writeTVar`, y que quien usa las operaciones las envuelve en `atomically`. En ese sentido, programar con STM se parece a la granularidad gruesa: es igual de simple, y la correctitud no depende de pensar cuidadosamente en qué orden tomar los locks.

Dos eliminaciones concurrentes. En la teórica vimos que, con un lock por nodo, no alcanzaba con que cada thread bloqueara sólo a `pred`. Con la lista `a -> b -> c -> d`, si un thread saca `b` (bloqueando `a`) y otro saca `c` (bloqueando `b`), el primero hace que `a.next` apunte a `c`, que el segundo acaba de sacar, y la eliminación de `c` se pierde. ¿Qué pasa con STM? La transacción que saca `b` *lee* `b.next` (para saber a dónde tiene que apuntar `a.next`), y la que saca `c` *escribe* `b.next`. Entonces las dos transacciones entran en conflicto, una de ellas se vuelve a ejecutar sobre la lista ya modificada, y no se pierde nada. No tuvimos que pensar este caso: alcanza con que cada `next` que el algoritmo usa haya sido leído dentro de la transacción. La prueba de `main` ejercita exactamente esta situación.

Se parece a la sincronización optimista. La estrategia optimista era: recorrer sin locks, bloquear `pred` y `curr`, validar que no haya cambiado nada importante y, si cambió, volver a empezar. STM hace algo muy parecido, pero lo hace el runtime en lugar del programador: la transacción recorre sin bloquear a nadie, al confirmarse se valida que ninguna de las `TVar`s leídas haya cambiado y, si alguna cambió, se vuelve a ejecutar desde el principio. El punto de linealización de cada operación es el momento en que su transacción se confirma.

La diferencia es que la validación de STM es más conservadora. En la versión optimista sólo importaba que `pred` siguiera siendo alcanzable y que `pred.next` siguiera siendo `curr`. En cambio, una transacción de STM leyó *todos* los punteros desde el principio de la lista, así que cualquier escritura en ese tramo la obliga a reintentar, aunque no la afecte. Por ejemplo, si un thread agrega un elemento cerca del principio mientras otro busca uno cerca del final, el segundo se vuelve a ejecutar. Por eso esta lista admite menos concurrencia que las versiones optimista y *lazy*: las operaciones avanzan en paralelo, pero las escrituras cerca del principio de la lista hacen reintentar a las que recorrieron ese tramo.

`contains`. En la versión *lazy*, `contains` no tomaba ningún lock. Acá `contains` es una transacción que sólo lee, así que nunca hace reintentar a otra transacción. Varios `contains` simultáneos no entran en conflicto entre sí. Pero sí se puede reintentar si una escritura de otro thread modifica un puntero que ya leyó.

Progreso. Como no hay locks, no puede haber deadlock. Pero tampoco hay garantía de que cada operación termine: una transacción larga (por ejemplo, una búsqueda cerca del final de una lista muy grande) puede tener que reintentar una y otra vez si otros threads modifican constantemente el principio de la lista. Recordemos que la granularidad fina con *hand-over-hand* sí era libre de inanición.

Lo que las versiones con locks no permiten: componer

Supongamos que tenemos dos conjuntos y queremos mover un elemento de uno al otro, de forma que ningún otro thread pueda ver un estado intermedio en el que el elemento no esté en ninguno de los dos. Con cualquiera de las versiones con locks de la teórica, no podemos construir esto a partir de `add` y `remove`: cada operación toma y suelta sus propios locks, y entre las dos queda un momento en el que el elemento no está en ningún conjunto. Para lograrlo tendríamos que conocer los locks internos de las dos listas y tomarlos desde afuera, con el riesgo de deadlock que eso implica.

Con STM, como `add` y `remove` devuelven transacciones, alcanza con combinarlas en una transacción más grande:

```
move :: List -> List -> Int -> STM Bool
move from to key = do
  removed <- remove from key
  if removed
    then do
      add to key
      return True
    else return False
```

y usarla con `atomically (move list1 list2 7)`. Toda la operación se confirma o se reintenta como una unidad, y no tuvimos que tocar el código de `add` ni de `remove`. Esta es la razón por la que conviene

que las operaciones de una estructura de datos devuelvan `STM` y no `IO`: el que las usa puede decidir qué operaciones forman parte de la misma transacción.